Windows → Kali Linux

File Transfer Techniques — Practical Lab Guide

PowerShell • SCP/SFTP • HTTP upload/download • SMB • Netcat • FTP • Shared folders • Troubleshooting

Purpose: A practical reference for transferring files from a Windows machine to Kali Linux in a local lab, authorized pentest, or HTB/OSCP environment.

Important: Use these methods only on systems and networks you own or are authorized to test. Confirm the correct interface/IP, firewall rules, and destination path before transferring files.

Commands use placeholders such as <KALI_IP>, <WINDOWS_FILE>, and <USERNAME>. Replace them with your actual values.

1. Choose the right method

Method	Best use	Needs
SCP	Simple secure copy; good default	SSH server on Kali; TCP/22 reachable
SFTP	Interactive secure file transfer	SSH server on Kali
HTTP upload server	PowerShell upload without SSH	Custom upload handler on Kali; TCP/8000
HTTP download	Kali serves files for Windows to download	HTTP server on Kali; TCP/8000
SMB share	Windows-native file sharing / shared folders	SMB server/share + credentials; TCP/445
Netcat	Quick one-off transfer in a trusted lab	Listener/connection; TCP port allowed
FTP	Legacy environments / compatibility	FTP server; preferably isolated lab only
VM shared folder	Windows host ↔ Kali VM on same computer	VMware shared folders enabled
Base64 over terminal	Tiny text files when network transfer isn't possible	Shell/clipboard; inefficient for binaries

Recommended order: SCP/SFTP first; HTTP upload if SSH is unavailable; SMB for Windows-centric workflows; Netcat only for a trusted lab. For a Windows host running Kali in VMware, shared folders may be simplest.

2. First checks: IP, route, and ports

On Kali, identify the interface and address that Windows can reach:

```
ip -br addr
ip route
ip -br addr show tun0   # only if using a VPN interface
```

On Windows PowerShell, test reachability (replace the address):

```
Test-NetConnection <KALI_IP> -Port 22
Test-NetConnection <KALI_IP> -Port 8000
Test-NetConnection <KALI_IP> -Port 445
```

A successful ping is not required for TCP transfer to work. A port test can fail because the service is stopped, the firewall blocks it, or the chosen IP belongs to an unreachable interface.

Avoid using a Kali VPN IP for a Windows host transfer unless Windows has a route to that VPN interface. For VMware, check the VM's NAT/bridged/host-only network mode.

3. Method A — SCP (recommended)

SCP copies a file over SSH and encrypts the transfer. On Kali:

```
sudo systemctl enable --now ssh
ip -br addr
```

From Windows PowerShell (Windows 10/11 OpenSSH Client is often available):

```
scp "C:\Users\Public\file.txt" kali@<KALI_IP>:/home/kali/
```

For a directory, add -r:

```
scp -r "C:\Users\Public\evidence" kali@<KALI_IP>:/home/kali/
```

If the command is missing, check `Get-Command scp` and Windows Optional Features → OpenSSH Client. On Kali, ensure the username and destination directory are correct.

Verify on Kali:

```
ls -lh /home/kali/file.txt
sha256sum /home/kali/file.txt
```

Tip: If you use a different Kali account, replace `kali` and destination path. Do not expose SSH to untrusted networks unnecessarily.

4. Method B — SFTP

SFTP uses the SSH service, but offers an interactive session. Start SSH on Kali, then run on Windows PowerShell:

```
sftp kali@<KALI_IP>
```

At the sftp prompt:

```
put "C:\Users\Public\file.txt" /home/kali/file.txt
put -r "C:\Users\Public\evidence" /home/kali/evidence
ls
exit
```

Use `put` to upload from Windows to Kali; `get` downloads from the remote server to Windows.

5. Method C — HTTP upload from PowerShell

Python's built-in `python3 -m http.server` serves downloads only; it does not accept POST uploads. For a simple controlled lab, use this minimal upload receiver on Kali. It writes files to /tmp and should not be exposed to untrusted networks.

```
cat > upload_server.py <<'PY'
from http.server import BaseHTTPRequestHandler, HTTPServer
from pathlib import Path

DEST = Path("/tmp/transfer")
DEST.mkdir(parents=True, exist_ok=True)

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        name = Path(self.path.lstrip("/")).name
        if not name or name in (".", ".."):
            self.send_error(400)
            return
        length = int(self.headers.get("Content-Length", "0"))
        if length <= 0 or length > 100 * 1024 * 1024:
            self.send_error(413)
            return
        data = self.rfile.read(length)
        (DEST / name).write_bytes(data)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"Upload OK")
    def log_message(self, fmt, *args):
        print("%s - %s" % (self.client_address[0], fmt % args))

HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
PY
python3 upload_server.py
```

From Windows PowerShell:

```
$file = "C:\Users\Public\file.txt"
$kaliIP = "<KALI_IP>"
Invoke-WebRequest -Uri "http://${kaliIP}:8000/file.txt" `
  -Method POST -InFile $file
```

The file is saved to `/tmp/transfer/file.txt` on Kali. The sample has no authentication or TLS; use only on a trusted isolated lab network and stop it with Ctrl+C after transfer.

6. Method D — HTTP download (Kali → Windows is the normal direction)

This method is useful when Kali needs to provide a file for Windows to download, not for uploading a Windows file to Kali.

On Kali, from the folder containing the file:

```
cd /tmp
python3 -m http.server 8000 --bind 0.0.0.0
```

On Windows PowerShell:

```
Invoke-WebRequest "http://<KALI_IP>:8000/file.txt" -OutFile "$env:TEMP\file.txt"
```

To transfer Windows → Kali, use the HTTP upload receiver in Method C instead.

7. Method E — SMB share

SMB is useful for Windows-style file sharing. One approach is to run a temporary SMB server on Kali with Impacket, if installed. This example makes the Windows machine connect to Kali's share and copy a file into it.

```
# Kali: create a destination directory
mkdir -p /tmp/smbdrop
# If impacket-smbserver is installed:
impacket-smbserver share /tmp/smbdrop -smb2support -username labuser -password 'Use-A-Strong-Lab-Password'
```

On Windows PowerShell, copy to the share (credentials depend on the SMB server setup):

```
net use \\<KALI_IP>\share /user:labuser Use-A-Strong-Lab-Password
Copy-Item "C:\Users\Public\file.txt" "\\<KALI_IP>\share\file.txt"
net use \\<KALI_IP>\share /delete
```

SMB server command-line options vary by Impacket version. Use the command's help if flags differ. TCP/445 must be reachable. Avoid exposing a temporary SMB server outside a lab.

8. Method F — Netcat (quick lab transfer)

Netcat has no built-in encryption or authentication. Use only on a trusted lab network. Start a receiver on Kali:

```
nc -lvnp 9001 > /tmp/file.txt
```

From Windows, if Ncat (bundled with Nmap) is installed:

```
ncat <KALI_IP> 9001 < "C:\Users\Public\file.txt"
```

Stop the listener after the transfer. Verify file size and hash on both sides. A connection reset or truncated file means the transfer may be incomplete; do not assume success solely because the command returned.

9. Method G — FTP (legacy; avoid on untrusted networks)

FTP transmits credentials and data without encryption by default. Use only in an isolated lab. Start a properly configured FTP server on Kali and connect with an FTP client from Windows; use the client's upload command (`put`). Server setup varies, so SCP/SFTP is generally simpler and safer.

If a legacy environment mandates FTP, use a dedicated temporary account, restrict the server to a dedicated directory, firewall it to the lab subnet, and stop it immediately after use.

10. Method H — VMware shared folders

If Windows is the host and Kali is a VMware guest on the same computer, a shared folder avoids network routing entirely.

Typical workflow:

```
# On Windows: configure a shared folder in VMware VM settings.
# In Kali, check whether it is mounted:
vmware-hgfsclient
ls /mnt/hgfs
```

If `/mnt/hgfs` is missing, VMware Tools / open-vm-tools and shared-folder support may need setup. Exact steps depend on VMware version and guest integration. Copy the file from the mounted share into your Kali home directory.

Treat shared folders as a deliberate trust boundary: files written by either OS may be visible to the other.

11. Method I — Base64 for tiny text files

Use only for small text/config snippets when no transfer channel is available. It is inefficient and can corrupt large/binary files.

On Windows PowerShell, encode a file:

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("C:\Users\Public\small.txt"))
```

Copy the resulting Base64 text to Kali, save it as a single line, then decode:

```
base64 -d small.txt.b64 > small.txt
```

```
sha256sum small.txt
```

For binaries, large files, or reliable evidence transfer, use SCP/SFTP/SMB instead.

12. Verification and troubleshooting

Always verify the transferred file. Compare SHA-256 hashes on both systems when integrity matters.

```
# Kali
sha256sum /path/to/file

# Windows PowerShell
Get-FileHash "C:\path\to\file" -Algorithm SHA256
```

Symptom	Checks
Connection timed out	Correct IP/interface; routing; host firewall; target port reachable.
Connection refused	Service is not listening, or wrong port.
Permission denied (SCP)	Correct Kali username/password/key; destination directory write permission.
HTTP upload gets 501/404	Built-in http.server does not accept uploads; run the upload receiver.
SMB access denied	Check share name, credentials, SMB dialect, and directory permissions.
File is empty/truncated	Check transfer exit status, source size, destination size, and SHA-256.
VMware shared folder absent	Check shared-folder setting and guest tools/service.

Quick recommendation: Same Windows host + Kali VM → VMware shared folder or SCP. Remote Windows → Kali → SCP/SFTP if SSH is reachable. If SSH is unavailable but HTTP is allowed, use a temporary lab-only upload receiver. Netcat/FTP are fallbacks, not secure defaults.

End of guide.